

✓ Simple Linear Regression: Marketing ROI Analysis

Analytical Optimization & Statistical Validation Pipeline

Project Objective: This notebook builds an Ordinary Least Squares (OLS) Simple Linear Regression model to identify which single marketing channel (TV, Radio, or Social Media) best predicts Sales revenue. It verifies classical linear regression assumptions using visual diagnostics and formal statistical tests before generating a clear, evidence-based budget recommendation.

Imports & Environment Setup

```
# =====
# Simple Linear Regression: Marketing ROI Analysis
# Analytical Optimization & Statistical Validation Pipeline
# File: regression_analysis.py
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import scipy.stats as stats
from statsmodels.stats.diagnostic import het_breuschpagan

# Set plotting style for crisp, clear visuals
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
print('Libraries and environment configurations loaded successfully.\n')
```

Libraries and environment configurations loaded successfully.

Step 1 – Load & Explore the Dataset

✓ 1. Load and Clean the Dataset

We import the tracking data, assess missing value density, and handle null fields to preserve the statistical power of the continuous sample.

Data Loading & Imputation

```
/content/drive/MyDrive/marketing_and_sales_data_evaluate_lr.csv.csv
```

```
# -----
# Step 1 - Load & Clean the Dataset
# -----
print("=== 1. Load and Clean the Dataset ===")
# Dynamically references the local workspace file name
file_path = '/content/drive/MyDrive/marketing_and_sales_data_evaluate_lr'

try:
    df = pd.read_csv(file_path)
    print("--- Dataset Initial Info ---")
    print(df.info())
    print("\nMissing Values Count Per Feature: ")
```

```

print("\n--- Missing Values Count Per Feature ---")
print(df.isnull().sum())

# Handle missing values by dropping rows (isolated strategy for simple regression)
df_clean = df.dropna().copy()
print("\n--- Summary Post-Cleaning ---")
print(f"Original Records: {df.shape[0]} | Cleaned Records Available: {df_clean.shape[0]}\n")
except FileNotFoundError:
    print(f"Error: Target data file '{file_path}' not located in active path.")
    exit()

```

=== 1. Load and Clean the Dataset ===

--- Dataset Initial Info ---

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4572 entries, 0 to 4571

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	TV	4562 non-null	float64
1	Radio	4568 non-null	float64
2	Social_Media	4566 non-null	float64
3	Sales	4566 non-null	float64

dtypes: float64(4)

memory usage: 143.0 KB

None

--- Missing Values Count Per Feature ---

TV 10

Radio 4

Social_Media 6

Sales 6

dtype: int64

--- Summary Post-Cleaning ---

Original Records: 4572 | Cleaned Records Available: 4546

```

# -----
# Step 2 – Exploratory Data Analysis & Feature Selection
# -----
print("=== 2. Exploratory Data Analysis & Feature Selection ===")
# 1. Output basic numeric parameters
print("--- Summary Statistics ---")
print(df_clean.describe())

# 2. Extract and visualize the correlation footprint
correlation_matrix = df_clean.corr()
print("\n--- Feature Correlation Matrix ---")
print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='Blues', fmt=".2f", vmin=-1, vmax=1)
plt.title('Correlation Matrix: Marketing Channels vs. Sales Summary', fontsize=12)
plt.savefig('correlation_matrix_edu.png', bbox_inches='tight')
plt.close()

# 3. Generate baseline regressions plots across all potential predictors
sns.pairplot(df_clean, x_vars=['TV', 'Radio', 'Social_Media'], y_vars='Sales', height=5, aspect=0.8)
plt.suptitle('Sales Distributions vs. Active Marketing Budgets with Regression Fits', y=1.05)
plt.savefig('pairplot_marketing_channels.png', bbox_inches='tight')
plt.close()

# 4. Programmatic choice of best predictor feature
target_variable = 'Sales'
predictors = [col for col in df_clean.columns if col != target_variable]
best_predictor = df_clean[predictors].corrwith(df_clean[target_variable]).abs().idxmax()

print("\n" + "="*80)
print(f"[SELECTION DECISION]: '{best_predictor}' exhibits the strongest correlation with Sales.")

```

```
print(f"Proceeding with Simple Linear Regression using Independent Vector: {best_predictor}")
print("=*80 + "\n")
```

```
=== 2. Exploratory Data Analysis & Feature Selection ===
```

```
--- Summary Statistics ---
```

	TV	Radio	Social_Media	Sales
count	4546.000000	4546.000000	4546.000000	4546.000000
mean	54.062912	18.157533	3.323473	192.413332
std	26.104942	9.663260	2.211254	93.019873
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.555355	1.530822	112.434612
50%	53.000000	17.859513	3.055565	188.963678
75%	77.000000	25.640603	4.804919	272.324236
max	100.000000	48.871161	13.981662	364.079751

```
--- Feature Correlation Matrix ---
```

	TV	Radio	Social_Media	Sales
TV	1.000000	0.869158	0.527687	0.999497
Radio	0.869158	1.000000	0.606338	0.868638
Social_Media	0.527687	0.606338	1.000000	0.527446
Sales	0.999497	0.868638	0.527446	1.000000

```
=====
[SELECTION DECISION]: 'TV' exhibits the strongest correlation with Sales.
Proceeding with Simple Linear Regression using Independent Vector: TV
=====
```

```
# -----
# Step 3 – OLS Regression Model Fitting
# -----
print("=== 3. OLS Regression Model Fitting ===")
# Structure arrays
X = df_clean[[best_predictor]]
y = df_clean[target_variable]

# Explicitly add the intercept column for OLS math
X_with_constant = sm.add_constant(X)

# Fit the OLS model
model = sm.OLS(y, X_with_constant).fit()

# View the full statistical balance sheet
print(model.summary())
print("\n")

# -----
# Step 3 – OLS Regression Model Fitting
# -----
print("=== 3. OLS Regression Model Fitting ===")
# Structure arrays
X = df_clean[[best_predictor]]
y = df_clean[target_variable]

# Explicitly add the intercept column for OLS math
X_with_constant = sm.add_constant(X)

# Fit the OLS model
model = sm.OLS(y, X_with_constant).fit()

# View the full statistical balance sheet
print(model.summary())
print("\n")
```

```
=== 3. OLS Regression Model Fitting ===
OLS Regression Results
```

```

=====
Dep. Variable:          Sales    R-squared:                0.999
Model:                  OLS      Adj. R-squared:           0.999
Method:                 Least Squares    F-statistic:             4.517e+06
Date:                  Thu, 18 Jun 2026    Prob (F-statistic):       0.00
Time:                  12:57:39    Log-Likelihood:          -11366.
No. Observations:      4546    AIC:                     2.274e+04
Df Residuals:          4544    BIC:                     2.275e+04
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1325	0.101	-1.317	0.188	-0.330	0.065
TV	3.5615	0.002	2125.272	0.000	3.558	3.565

```

=====
Omnibus:                0.052    Durbin-Watson:           1.998
Prob(Omnibus):           0.974    Jarque-Bera (JB):         0.031
Skew:                   -0.001    Prob(JB):                 0.985
Kurtosis:               3.012    Cond. No.                 138.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

=== 3. OLS Regression Model Fitting ===

OLS Regression Results

```

=====
Dep. Variable:          Sales    R-squared:                0.999
Model:                  OLS      Adj. R-squared:           0.999
Method:                 Least Squares    F-statistic:             4.517e+06
Date:                  Thu, 18 Jun 2026    Prob (F-statistic):       0.00
Time:                  12:57:39    Log-Likelihood:          -11366.
No. Observations:      4546    AIC:                     2.274e+04
Df Residuals:          4544    BIC:                     2.275e+04
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.1325	0.101	-1.317	0.188	-0.330	0.065
TV	3.5615	0.002	2125.272	0.000	3.558	3.565

```

=====
Omnibus:                0.052    Durbin-Watson:           1.998
Prob(Omnibus):           0.974    Jarque-Bera (JB):         0.031
Skew:                   -0.001    Prob(JB):                 0.985
Kurtosis:               3.012    Cond. No.                 138.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

# -----
# Step 4 – Core OLS Statistical Assumption Diagnostics
# -----
print("=== 4. OLS Regression Diagnostic Dashboard ===")
fitted_values = model.fittedvalues
residuals = model.resid

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Panel A: Linearity Check (Residuals vs Fitted Values)
sns.scatterplot(x=fitted_values, y=residuals, ax=axes[0], alpha=0.7, color='indigo')
axes[0].axhline(y=0, color='red', linestyle='--')
axes[0].set_title('Linearity Evaluation:\nResiduals vs. Fitted Values')
axes[0].set_xlabel('Predicted/Fitted Sales Values')
axes[0].set_ylabel('Residual Error')

```

```
# Panel B: Normality Verification via Quantile-Quantile Plot
stats.probplot(residuals, dist="norm", plot=axes[1])
axes[1].get_lines()[0].set_color('indigo')
axes[1].get_lines()[0].set_alpha(0.6)
axes[1].set_title('Normality Evaluation:\nQuantile-Quantile (Q-Q) Plot')

# Panel C: Homoscedasticity Verification via Scale-Location Dashboard
model_norm_residuals = model.get_influence().resid_studentized_internal
sns.scatterplot(x=fitted_values, y=np.sqrt(np.abs(model_norm_residuals)), ax=axes[2], alpha=0.7,
axes[2].set_title('Homoscedasticity Evaluation:\nScale-Location Dashboard')
axes[2].set_xlabel('Predicted/Fitted Sales Values')
axes[2].set_ylabel(r'$\sqrt{|\text{Standardized Residuals}|}$')

plt.tight_layout()
plt.savefig('regression_diagnostic_dashboard.png', bbox_inches='tight')
plt.close()

# --- Formal Statistical Hypothesis Tests ---
print("--- Diagnostic Tests Summary ---")

# Jarque-Bera Normality Test
jb_stat, jb_p = stats.jarque_bera(residuals)
print(f"Jarque-Bera Normality Test p-value: {jb_p:.4e}")
if jb_p > 0.05:
    print(" -> Result: Fail to reject H0. Residual errors are normally distributed.")
else:
    print(" -> Result: Reject H0. Errors show significant non-normality.")

# Breusch-Pagan Homoscedasticity Test
bp_stat, bp_p, _, _ = het_breuschpagan(residuals, X_with_constant)
print(f"Breusch-Pagan Homoscedasticity Test p-value: {bp_p:.4e}")
if bp_p > 0.05:
    print(" -> Result: Fail to reject H0. Homoscedasticity confirmed (constant variance).")
else:
    print(" -> Result: Reject H0. Heteroscedasticity detected; variance scales with prediction.")
print("\n")
```

```
=== 4. OLS Regression Diagnostic Dashboard ===
--- Diagnostic Tests Summary ---
Jarque-Bera Normality Test p-value: 9.8458e-01
-> Result: Fail to reject H0. Residual errors are normally distributed.
Breusch-Pagan Homoscedasticity Test p-value: 9.9386e-01
-> Result: Fail to reject H0. Homoscedasticity confirmed (constant variance).
```

```
# -----
# Step 5 – Strategic ROI Formulation
# -----

r_squared = model.rsquared
intercept = model.params.iloc[0]
slope = model.params.iloc[1]
p_value = model.pvalues.iloc[1]

print("="*85)
print("                                STRATEGIC BUSINESS ROI INTERPRETATION REPORT                                ")
print("="*85)
print(f"1. Explanatory Predictive Power (R-squared): {r_squared:.4f}")
print(f" - Approximately {r_squared*100:.1f}% of the variance in Sales is explained by your {best_predictor}")
print("\n2. Statistical Confidence Profile:")
print(f" - The calculated p-value for the {best_predictor} spending coefficient is {p_value:.4e}.")
print(f" - Conclusion: This feature is highly significant (p < 0.05), proving real structural dependency.")
print("\n3. Quantifiable ROI Efficiency Blueprint:")
print(f" - Definitive Empirical Model Equation: Sales = ({slope:.4f} * {best_predictor}) + ({intercept:.4f})")
print(f" - Base System Sales Constant (Intercept): {intercept:.4f} units generated when channel budget is zero")
print(f" - Definitive Channel Efficiency (Slope): For every additional $1.00 injected into {best_predictor},")
print(f" overall Sales volume scales upwards by exactly {slope:.4f} units.")
```

```
print("="*85)
print(f"EXECUTIVE RECOMMENDATION: Aggressively prioritize budget expansions toward '{best_predictor}'")
print("="*85)
```

```
=====
                        STRATEGIC BUSINESS ROI INTERPRETATION REPORT
=====
1. Explanatory Predictive Power (R-squared): 0.9990
   - Approximately 99.9% of the variance in Sales is explained by your TV budget.

2. Statistical Confidence Profile:
   - The calculated p-value for the TV spending coefficient is 0.0000e+00.
   - Conclusion: This feature is highly significant (p < 0.05), proving real structural dependency

3. Quantifiable ROI Efficiency Blueprint:
   - Definitive Empirical Model Equation: Sales = (3.5615 * TV) + (-0.1325)
   - Base System Sales Constant (Intercept): -0.1325 units generated when channel budgets are zero
   - Definitive Channel Efficiency (Slope): For every additional $1.00 injected into TV,
     overall Sales volume scales upwards by exactly 3.5615 units.

=====
EXECUTIVE RECOMMENDATION: Aggressively prioritize budget expansions toward 'TV'.
=====
```